Lernplan Masterarbeit — Was muss ich lernen & erarbeiten?

Stand: 24.02.2026

BLOCK A: KONZEPTIONELLE GRUNDLAGEN (vor dem Coding)

A1. EU AI Act — Artikel 9–15 im Detail

Was: Jeden Artikel einzeln lesen, verstehen, technische Implikationen ableiten **Warum:** Ohne das kannst du keine Compliance-Gates definieren **Status:** ⬜ Noch nicht erarbeitet

Artikel	Thema	Technische Implikation	Lernstatus
Art. 9	Risikomanagementsystem	Risiko-Assessment als Gate-Kriterium, kontinuierliche Überwachung	⬜
Art. 10	Daten-Governance	Datenqualitäts-Checks, Bias-Prüfung, Trainingsdaten-Dokumentation	⬜
Art. 11	Technische Dokumentation	Model Cards, Architektur-Doku, automatisierte Doku-Generierung	⬜
Art. 12	Aufzeichnungspflichten (Logging)	Immutable Logs, Audit Trail, Evidence Store	⬜
Art. 13	Transparenz	Erklärbarkeit, User-Facing Dokumentation	⬜
Art. 14	Menschliche Aufsicht	Human-in-the-Loop Gates, Eskalationsprozesse	⬜
Art. 15	Genauigkeit, Robustheit, Cybersicherheit	Eval-Metriken, Adversarial Testing, Security Scans	⬜

Lerninstanzen:

- Primärquelle: EU AI Act Volltext (eur-lex.europa.eu) — Art. 9–15 sind ca. 15 Seiten
- Sovrano et al. (2025) — aus deinem Literaturverzeichnis, analysiert technische Implikationen
- Taeihagh (2025) — Governance-Perspektive
- Claude: Artikel hochladen → pro Artikel fragen "Was muss technisch geprüft werden?"

A2. Anforderungskatalog (R1–Rn) ableiten

Was: Aus 3 Quellen systematisch Anforderungen extrahieren **Warum:** Ohne Anforderungen keine Design-Prinzipien, ohne Prinzipien keine Architektur **Status:** ⬜ Noch nicht erarbeitet

Quelle	Anforderungstyp	Methode	Lernstatus

Quelle	Anforderungstyp	Methode	Lernstatus
Literatur (PD1/PD2/PD3)	Funktionale & nicht-funktionale Req.	Paper re-lesen, Anforderungen extrahieren	⬜
EU AI Act (Art. 9–15)	Compliance-Anforderungen	Regulatory-to-Technical Mapping (A1)	⬜
CNCF/SLSA Best Practices	Technische Anforderungen	SLSA Levels, CNCF Guidelines lesen	⬜

Lerninstanzen:

- Kreuzberger et al. (2023) — MLOps Anforderungen (bereits in Zotero)
- SLSA Specification: slsa.dev/spec — Levels 1–4 durchlesen
- CNCF Cloud Native Definition: github.com/cncf/toc
- Claude: "Hier sind meine 3 Quellen → extrahiere Anforderungen als Tabelle"

A3. Gate-Katalog spezifizieren

Was: Für jede PoC-Phase (EVAL, DEPLOY, PROD) konkrete Gates definieren **Warum:** Das ist USP 1 — dein Kernbeitrag **Status:** ⬜ Noch nicht erarbeitet

Pro Gate brauchst du:

- Trigger (wann feuert das Gate?)
- Kriterien (was wird geprüft? Schwellenwerte?)
- Evidenzartefakte (was wird als Beweis gespeichert?)
- Entscheidungslogik (Pass/Fail/Waiver?)
- Owner (wer ist verantwortlich?)
- Audit Trail (wie wird die Entscheidung dokumentiert?)

Beispiel-Gates die du definieren musst:

Phase	Gate-Name	Dimension	Beispiel-Kriterium
EVAL	Hallucination Check	Technisch	Faithfulness Score > 0.85 (DeepEval)
EVAL	Bias Assessment	Compliance (Art. 10)	Bias-Metriken unter Schwellenwert
DEPLOY	Model Card Completeness	Compliance (Art. 11)	Alle Pflichtfelder ausgefüllt
DEPLOY	Supply Chain Signature	Technisch	Sigstore-Signatur vorhanden + gültig
DEPLOY	OPA Policy Check	Compliance (Art. 9)	Risiko-Assessment dokumentiert
PROD	Drift Detection	Technisch	Output-Verteilung innerhalb Toleranz
PROD	Logging Completeness	Compliance (Art. 12)	Alle Interaktionen im Evidence Store
PROD	Human Oversight Trigger	Compliance (Art. 14)	Eskalation bei Confidence < Threshold

Lerninstanzen:

- DeepEval Docs: docs.confident-ai.com — Metriken verstehen (Faithfulness, Hallucination, etc.)
- OPA Docs: openpolicyagent.org/docs — wie Rego-Policies funktionieren
- Keine externe Quelle für Gate-Katalog selbst — DAS ist deine Eigenleistung

---

A4. Design-Prinzipien formulieren

Was: Aus Anforderungen (A2) abstrakte Prinzipien ableiten **Warum:** Prinzipien = Brücke zwischen "was" (Anforderungen) und "wie" (Architektur) **Status:** ⬜ Noch nicht erarbeitet

Erwartete Prinzipien (Hypothese, muss aus A2 abgeleitet werden):

- DP1: Trennung von Policy und Enforcement
- DP2: Immutable Evidence
- DP3: Git als Single Source of Truth
- DP4: Cloud-Agnostik auf Referenz-Ebene
- DP5: Compliance-as-Code
- DP6: Continuous Verification (nicht nur einmalig)

Lerninstanzen:

- Hevner et al. (2004) — DSR Design-Prinzipien Methodik
- March & Smith (1995) — Artefakttypen in DSR
- Gregor & Hevner (2013) — "Positioning and Presenting Design Science Research"

BLOCK B: TECHNISCHE FÄHIGKEITEN (für den PoC)

B1. Kubernetes (AKS) — Grundlagen

Was: Cluster erstellen, Deployments, Services, Namespaces, RBAC verstehen **Warum:** Deine Architektur läuft auf K8s **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- Microsoft Learn: "Introduction to AKS" (kostenlos, mit Sandbox)
- Kubernetes Basics: kubernetes.io/docs/tutorials/kubernetes-basics/
- Hands-on: Azure Student Subscription → AKS Cluster erstellen
- Dauer: ca. 1–2 Wochen bei 2h/Tag

B2. Terraform / IaC

Was: Azure-Ressourcen als Code definieren, Plan/Apply-Zyklus, Module **Warum:** Säule 1 — Infrastruktur wird nicht geklickt, sondern codiert **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- HashiCorp Learn: learn.hashicorp.com (kostenlose Tutorials)
- Microsoft Learn: "Deploy Azure infrastructure with Terraform"
- Hands-on: AKS + Azure Blob Storage mit Terraform aufsetzen

- Dauer: ca. 1 Woche bei 2h/Tag

B3. OPA / Rego — Policy-as-Code

Was: Rego-Syntax, Policies schreiben, OPA in K8s integrieren (Gatekeeper) **Warum:** Säule 2 + 4 — Kernkomponente für Gates UND Compliance **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- OPA Playground: play.openpolicyagent.org (sofort im Browser)
- Styra Academy: academy.styra.com (kostenlos, strukturiert)
- OPA Docs: openpolicyagent.org/docs/latest/
- Hands-on: 3 Policies schreiben (1× Deployment-Gate, 1× AI-Act Art. 9, 1× Art. 12)
- Dauer: ca. 1–2 Wochen bei 2h/Tag

B4. DeepEval — LLM Evaluation

Was: Test-Suites für LLM-Output, Metriken (Faithfulness, Hallucination, Toxicity) **Warum:** Säule 2 — technische Gate-Kriterien **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- DeepEval Docs: docs.confident-ai.com
- GitHub: github.com/confident-ai/deepeval — Examples-Ordner
- Hands-on: pytest + DeepEval auf einem einfachen LLM-Call laufen lassen
- Dauer: ca. 3–5 Tage bei 2h/Tag

B5. GitHub Actions — CI/CD Pipeline

Was: Workflows schreiben (.yml), Triggers, Jobs, Steps, Secrets, Matrix **Warum:** Säule 3 — Pipeline-Automatisierung **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- GitHub Docs: docs.github.com/en/actions (Learning Pathways)
- Microsoft Learn: "Build CI/CD with GitHub Actions"
- Hands-on: Pipeline bauen die DeepEval-Tests + OPA-Check + Deploy auslöst
- Dauer: ca. 1 Woche bei 2h/Tag

B6. ArgoCD — GitOps Deployment

Was: ArgoCD auf K8s installieren, App-of-Apps Pattern, Sync Policies **Warum:** Säule 3 — deklaratives Deployment, Git = SSoT **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- ArgoCD Docs: argo-cd.readthedocs.io/en/stable/getting_started/
- YouTube: "ArgoCD Tutorial for Beginners" (TechWorld with Nana)

- Hands-on: ArgoCD auf AKS deployen, eine App syncen
- Dauer: ca. 3–5 Tage bei 2h/Tag

B7. Sigstore / Cosign — Supply Chain Security

Was: Container-Images signieren, Signaturen verifizieren, SLSA Attestations **Warum:** Säule 3 — Beweis dass Artifacts nicht manipuliert wurden **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- Sigstore Docs: docs.sigstore.dev
- Cosign Quickstart: github.com/sigstore/cosign
- Hands-on: Ein Docker-Image signieren und verifizieren
- Dauer: ca. 2–3 Tage bei 2h/Tag

B8. Azure Blob Storage (Immutable) — Evidence Store

Was: Immutable Storage Policies, WORM (Write Once Read Many), Retention **Warum:** Säule 4 — Compliance-Evidenz darf nicht verändert werden **Aktuelles Level:** ⬜ Einschätzung nötig

Lerninstanzen:

- Microsoft Learn: "Store business-critical blob data with immutable storage"
- Hands-on: Blob Container mit Immutability Policy erstellen, Evidenz reinschreiben
- Dauer: ca. 1–2 Tage bei 2h/Tag

BLOCK C: WISSENSCHAFTLICHE METHODIK

C1. DSR sauber durchführen

Was: Hevner-Zyklen, Peffers-Schritte, vom Brocke DSR Grid — nicht nur zitieren, sondern anwenden **Warum:** Das ist dein methodischer Rahmen, der Prof wird fragen "wo sind Sie im DSR-Zyklus?" **Status:** 🟡 Exposé-Level vorhanden, Anwendung auf Kapitel 3–6 fehlt

Lerninstanzen:

- Hevner et al. (2004) — nochmal lesen mit Fokus auf Evaluation-Methoden
- Peffers et al. (2007) — DSRM-Schritte auf deine Arbeit mappen
- Gregor & Hevner (2013) — Knowledge Contribution Framework
- Beispiel-DSR-Thesis lesen (1–2 aus deinem Fachbereich, über Google Scholar suchen)

C2. Evaluation planen

Was: Wie bewertest du, ob deine Architektur "gut" ist? **Warum:** Ohne Evaluation keine DSR-Arbeit — das ist Kap. 6 **Status:** ⬜ Noch nicht erarbeitet

Evaluationsoptionen:

- Szenario-Walkthrough: Art. 9–15 als Testfälle durch die Architektur laufen lassen
- Expertenreview: 2–3 Experten bewerten Architektur + Gates
- PoC-Demonstration: Funktioniert die Pipeline end-to-end?
- Feature-Vergleich: Deine Architektur vs. existierende Frameworks

Lerninstanzen:

- Venable et al. (2016) — "FEDS: A Framework for Evaluation in Design Science Research"
- Prat et al. (2015) — Taxonomy of Evaluation Methods for IS Artifacts
- → Prof morgen fragen: "Welche Evaluationstiefe erwarten Sie?"

BLOCK D: ZEITLICHE EINORDNUNG

Empfohlene Reihenfolge:

1. **Sofort (diese Woche):** A1 (EU AI Act lesen) — Grundlage für alles
2. **Woche 2–3:** A2 (Anforderungskatalog) + A3 (Gate-Katalog Entwurf)
3. **Woche 3–4:** A4 (Design-Prinzipien) + C1 (DSR vertiefen)
4. **Woche 4–8:** B1–B8 (Technische Skills parallel zum Schreiben von Kap. 3–5)
5. **Woche 8–12:** PoC bauen (Kap. 6) mit Claude Code
6. **Woche 12–16:** Evaluation (C2) + Kap. 7 (Diskussion)
7. **Woche 16–20:** Schreiben, Revision, Feinschliff

Abhängigkeiten:

```
A1 (AI Act) ──→ A2 (Anforderungen) ──→ A4 (Prinzipien) ──→ Kap. 5
(Architektur)
                      ↓
                 A3 (Gate-Katalog) ──→ B3 (OPA) + B4 (DeepEval) ──→ Kap. 6
(PoC)

↓
                                                                    C2
(Evaluation)
```

OFFENE FRAGEN (für Prof-Gespräch morgen):

1. Evaluationstiefe: Szenario-Walkthrough reicht, oder funktionierender Prototyp?
2. Anforderungsquellen: Nur Literatur + AI Act, oder auch Experteninterviews?
3. Gate-Katalog: Wie viele Gates exemplarisch? 3–5 oder vollständig?
4. PoC-Umfang: Reicht ein Durchstich (1 Use Case durch alle Phasen)?
5. Zeitrahmen: Ist das Zeitbudget realistisch für den geplanten Scope?